PROJECT PROPOSAL

REAL-TIME VOICE CHANGER

A Java-Based Real-Time Audio Processing Application

JAVA	REAL-TIME DSP	10 VOICE PRESETS

Submitted To

Maharin Afroj

[Lecturer,CSE,BUBT]

Submitted By - Team of Four

No.	Team Member	Student ID	Primary Responsibility
1	Habib	177	Team Lead & Integration
2	Sadman	183	Audio Processing & Backend
3	Pranto	187	UI/UX & Frontend
4	Fahim	197	Testing & Documentation

Course: Advance Programming Lab | Course Code: CSE-342
Department of Computer Science and Engineering
Submission Date: 19/07/26

Proposal at a Glance

Project Concept: The proposed system will capture the user's microphone audio, apply a selected voice effect in real time, and send the processed sound to a virtual audio output so that people in voice calls, online meetings, games, or communication applications hear the modified voice.

Project Title	Real-Time Voice Changer
Project Type	Java desktop application with an optional web-style clickable interface
Target Platform	Windows desktop/laptop
Core Technology	Java Sound API and digital signal processing
Main Output	Real-time processed audio routed as a virtual microphone
Number of Presets	10 selectable voice effects
Team Size	4 members

1. Introduction

1.1 Background

Voice communication is now a common part of online classes, meetings, gaming, content creation, customer support, and social interaction. In many situations, users want to change the character of their voice for entertainment, privacy, role-playing, creative presentation, or accessibility. Most existing voice-changing tools are either paid, closed-source, difficult to customize, or dependent on complex software. Therefore, a lightweight and understandable Java-based solution can be a practical academic project that demonstrates audio input/output, real-time processing, user-interface design, and system integration.

1.2 Problem Statement

A normal microphone sends the original voice directly to communication software. The proposed project must intercept that microphone signal, transform it with a selected effect, and provide the changed audio to another application with minimum noticeable delay. The main technical challenges are real-time audio capture, low-latency processing, smooth switching between effects, output-device routing, and avoiding noise, feedback, or audio distortion.

1.3 Proposed Solution

The team proposes to develop a Real-Time Voice Changer using Java. The program will allow the user to select an input microphone, choose one of ten voice presets, start or stop processing, and route the modified audio to a virtual audio device. A modern JavaFX interface will be used as the primary clickable interface. If required, a lightweight HTML, CSS, and JavaScript interface may be added as an optional control panel, while Java remains responsible for the main audio-processing logic.

1.4 Project Objectives

- Capture live voice data from a microphone using Java.
- Process the audio continuously with low and stable latency.
- Provide ten clearly different voice-changing presets.
- Allow the user to select input and output audio devices.
- Route processed audio to calling or communication applications through a virtual audio cable.
- Create a clean, clickable, and easy-to-use interface.
- Apply modular object-oriented design so that new effects can be added later.
- Test the application for clarity, responsiveness, stability, and usability.

1.5 Scope of the Project

The initial version will focus on Windows computers and mono microphone input. It will support real-time operation, ten preset effects, device selection, start/stop controls, volume control, and optional microphone monitoring. The project will not attempt to create a new operating-system audio driver from scratch. Instead, it will use an available virtual audio cable to connect the Java application to Discord, Google Meet, Zoom, games, or similar software. Natural human voice conversion is a complex research problem; therefore, the project will provide convincing effect-based transformations rather than claiming perfect identity or gender conversion.

1.6 Working Principle



Audio Flow: input is captured, transformed, buffered, and routed as a virtual microphone.

1.7 Expected Outcome

At the end of the project, the team expects to deliver a working application in which a user can speak normally, select a preset, and make another person hear the processed voice through a supported communication application. The final submission will include the source code, executable build, configuration guide, testing report, screenshots, and project documentation.

1.8 Development Approach

Phase	Major Activities	Expected Result
Phase 1 - Planning	Confirm requirements, study Java audio devices, set up repository and tools.	Approved design and working development environment.
Phase 2 - Audio Engine	Implement microphone capture, buffering, output, and basic bypass mode.	Stable live audio pipeline.
Phase 3 - Effects and UI	Develop ten presets, device controls, sliders, and the JavaFX interface.	Feature-complete prototype.
Phase 4 - Integration and Testing	Connect virtual routing, tune latency, fix bugs, prepare documentation.	Demonstration-ready final application.

1.9 Key Risks and Mitigation

High latency	Tune buffer size, minimize unnecessary processing, and keep audio work on dedicated threads.
Feedback or echo	Recommend headphones and provide a monitoring on/off control.
Audio format mismatch	Validate sample rate, bit depth, and channel format before opening devices.
Effect instability	Use modular presets, clipping protection, and repeated switching tests.
Device routing confusion	Provide a setup guide showing the correct virtual input and output selections.

2. Required Technologies

The project combines Java programming, digital audio processing, interface development, and audio-device routing. The following technologies are proposed for the implementation.

Technology / Tool	Purpose in the Project	Reason for Selection
Java 17 or Java 21	Core application logic, object-oriented design, threading, and device control.	Stable long-term Java versions with modern language support.
Java Sound API	Captures microphone audio through TargetDataLine and sends processed audio through SourceDataLine.	Built into Java and suitable for basic real-time audio input/output.
TarsosDSP	Supports pitch shifting, time stretching, filters, delay, and other DSP operations.	Reduces the need to implement every audio algorithm from the beginning.
JavaFX	Creates the main graphical interface with buttons, sliders, device selectors, indicators, and waveform display.	Provides a professional desktop UI while keeping the project Java-based.
HTML, CSS, JavaScript (Optional)	Creates an optional browser-like or embedded control interface.	Useful when a web-style clickable presentation is preferred.
Maven or Gradle	Manages libraries, builds the project, and produces a reproducible application package.	Simplifies dependency and build management.
VB-CABLE or Equivalent	Receives processed output and exposes it as an input device to calling applications.	Enables practical virtual-microphone routing without developing a driver.
Git and GitHub	Version control, teamwork, issue tracking, backup, and code review.	Suitable for a four-member development team.
IntelliJ IDEA / VS Code	Source-code development, debugging, testing, and project management.	Common and efficient Java development environments.

2.1 Recommended Audio Configuration

Sample Rate	44.1 kHz or 48 kHz
Sample Size	16-bit PCM
Channels	Mono
Buffer Size	Approximately 256-1024 samples, tuned during testing

Target Latency	Low enough for conversation; preferably below approximately 150 ms
Output Routing	Selected physical output or virtual audio cable

2.2 Proposed Software Architecture

Layer	Main Components	Responsibility
Presentation Layer	JavaFX controls, optional HTML/CSS/JS panel	Receives user commands and displays status.
Control Layer	Controller, preset manager, device manager	Coordinates device selection, effect changes, and application state.
Audio Engine	Capture thread, processing chain, output thread	Moves audio safely and continuously through the system.
Effect Layer	Pitch, filters, modulation, delay, reverb, distortion	Transforms the live voice according to the selected preset.
Routing Layer	Physical speaker/headphone or virtual audio cable	Delivers the processed audio to the user or another application.

2.3 Development and Testing Environment

- Operating System: Windows 10 or Windows 11.
- Minimum Hardware: dual-core processor, 4 GB RAM, microphone, and headphones.
- Recommended Hardware: modern multi-core processor, 8 GB RAM, and a low-latency wired headset.
- Testing Applications: a local recording tool and at least one supported calling application.
- Headphones are recommended during testing to prevent speaker-to-microphone feedback.

3. Features

3.1 Core Functional Features

- Live microphone capture and continuous audio processing.
- Input microphone and output device selection.
- Ten one-click voice presets with instant switching.
- Start, stop, mute, reset, and bypass controls.
- Pitch, effect strength, and output-volume adjustment.
- Visual input and output level indicators.
- Optional local monitoring through headphones.
- Virtual microphone routing for external communication applications.
- Preset status display and basic error notifications.
- Configurable audio buffer for balancing quality and latency.

3.2 Ten Proposed Voice Presets

No.	Preset	Main Processing Method	Expected Sound
1	Deep Voice	Lower pitch, light bass enhancement, optional formant adjustment	Deeper and heavier voice
2	Bright / Female-Style Voice	Higher pitch, brightness control, formant adjustment where available	Lighter and brighter character
3	Child Voice	Higher pitch and faster tonal movement	Small, youthful, cartoon-like voice
4	Robot Voice	Ring modulation, metallic tone, mild distortion	Mechanical and electronic sound
5	Monster Voice	Lower pitch, bass boost, distortion, and reverb	Large, rough, frightening sound

No.	Preset	Main Processing Method	Expected Sound
6	Alien Voice	Pitch modulation, tremolo, short delay, and filtering	Unusual science-fiction sound
7	Radio Voice	Band-pass filter, compression, and slight noise	Walkie-talkie or broadcast effect
8	Echo Voice	Delay with adjustable feedback and wet/dry mix	Repeating voice reflections
9	Cave Voice	Longer reverb with low-frequency emphasis	Voice speaking inside a large cave
10	Demon Voice	Very low pitch, distortion, layered echo, and reverb	Dark, dramatic, supernatural sound

Technical Note: The Bright / Female-Style preset will be presented as an effect-based transformation. A natural voice conversion system normally requires advanced formant processing or machine-learning models, so the academic version will focus on a clear and convincing real-time result rather than perfect identity conversion.

3.3 User Interface Features

- A home screen with project title and system status.
- Drop-down menus for microphone and output-device selection.
- Ten clearly labelled preset buttons arranged in a grid.
- Sliders for pitch, strength, volume, echo, and reverb where appropriate.
- Start/Stop button with active-state indication.
- Live audio-level meter and optional waveform visualization.
- A help panel explaining virtual-audio routing and common troubleshooting steps.
- Responsive layout suitable for standard laptop screens.

3.4 Non-Functional Requirements

Requirement	Target	Verification Method
Latency	Conversational delay should remain low and stable.	Measure microphone-to-output delay during testing.
Audio Quality	Speech should remain understandable with no severe clipping.	Record samples and evaluate with team members.
Reliability	Application should continue processing without crashes.	Run extended test sessions and switch presets repeatedly.
Usability	A new user should start an effect with minimal instructions.	Conduct a short user test and collect feedback.
Maintainability	Each effect should be separated into reusable classes.	Review code structure and documentation.
Security and Privacy	Live voice should be processed locally by default.	Verify that no audio is uploaded without explicit design changes.

3.5 Team Responsibilities

Team Role	Main Tasks	Key Deliverables
Member 1 - Team Lead & Integration	Project planning, architecture, module integration, final build.	Integrated application and progress coordination.
Member 2 - Audio Processing	Microphone capture, DSP algorithms, presets, latency tuning.	Audio engine and effect modules.
Member 3 - UI/UX Development	JavaFX interface, styling, device controls, visual indicators.	Clickable user interface and usability improvements.
Member 4 - Testing & Documentation	Test cases, bug reporting, user guide, presentation materials.	Testing report, documentation, and final presentation.

3.6 Planned Deliverables

- Complete Java source code with comments and modular package structure.
- Executable application package and dependency configuration.
- Ten working voice presets and device-routing support.
- Installation and user guide with screenshots.
- Test report containing latency, audio-quality, and stability observations.
- Final project presentation and live demonstration.

3.7 Future Enhancement Opportunities

Possible future improvements include custom user-created presets, noise suppression, equalizer controls, recording and export, plugin support, mobile compatibility, AI-assisted voice conversion, multilingual voice profiles, and a native virtual microphone driver. These enhancements are outside the initial academic scope but demonstrate that the proposed architecture can be extended.

3.8 Final Acceptance Criteria

- The application detects at least one microphone and one usable output device.
- A user can activate, change, bypass, and stop effects without restarting the program.
- All ten presets produce clearly distinguishable audible results.
- Processed audio can be selected as an input in at least one calling application through virtual routing.
- The application completes an extended demonstration without crashing or losing the selected device.
- Installation, routing, and basic troubleshooting steps are documented.

3.9 Typical User Scenario

1. The user connects a microphone or headset and launches the application.
2. The user selects the physical microphone as input and the virtual cable as output.
3. The user chooses one of the ten presets and presses Start.
4. In the calling application, the user selects the corresponding virtual-cable output as the microphone.
5. The listener hears the processed voice, while the user can switch presets or bypass the effect at any time.

Conclusion

The Real-Time Voice Changer is a feasible and educational four-member Java project. It combines object-oriented programming, digital signal processing, multithreading, interface design, testing, and real-world system integration. By limiting the first version to ten well-defined effects and using an existing virtual audio cable for routing, the team can focus on building a stable, presentable, and demonstrable application within an academic project timeline.